py-flexplot :: Python Cheatsheet

Intelligent formula-based visualization — port of Dustin Fife's R package · powered by plotnine + statsmodels

Python

v0.3.0 · Updated: 2026-08

Basics

INSTALL

```
pip install git+https://github.com/
  ezraair555/py-flexplot.git

# Optional backends for flex_nn:
pip install torch
pip install "keras[jax]"
```

IMPORT

```
from pyflexplot import (
    flexplot, visualize,
    compare_fits, added_plot,
)
from pyflexplot.stats import (
    model_comparison, estimates,
    p_format, eliminated_columns,
    color_table,
)
from pyflexplot.bluepill import (
    estimate_sd, mixed_model
)
from pyflexplot.ebbr import (
    fit_beta_prior, add_ebb_estimate
)
from pyflexplot.sem import (
    hopper_plot, disturbance_plot,
    measurement_plot,
)
from pyflexplot.flex_nn import (
    NeuralNetFit, permutation_importance
)
```

FORMULA SYNTAX

```
"outcome ~ predictor + color | panel_col + panel_row"
```

Component	Role
`outcome`	Y-axis (must be numeric for most plots)
`predictor`	X-axis variable
`color`	2nd predictor → color/group
`|`	Separator for paneling
`panel_col`	1st given → column facets
`panel_row`	2nd given → row facets

Formula is a **string** in Python (not R formula object). Max 2 given vars.

FLEXPLOT() SIGNATURE

```
flexplot(formula: str, data: pd.DataFrame, method: str = "auto",
**kwargs)
```

method	Smoothing
`"auto"`	LM (default)
`"lm"`	Linear regression line
`"loess"`	Local polynomial smooth

Binary 0/1 outcome → binomial GLM auto-selected. Invalid method → `ValueError`.

QUICK EXAMPLES

```
import pandas as pd
df = pd.read_csv("data.csv")

# Scatterplot with panels
p = flexplot("y ~ x + z | g1 + g2",
             data=df, method="lm")
p.draw()
```

```
# Intercept-only → histogram
p = flexplot("y ~ 1", data=df)
```

```
# Categorical × numeric
p = flexplot("satisfaction ~ gender",
             data=df)
```

Returns a `plotnine.ggplot` object. Call `.draw()` to render in a notebook. Extend with `+` and any plotnine layer.

Model Visualization

VISUALIZE() — FITTED MODELS

```
visualize(model, data=None, **kwargs)
```

Auto-generates predicted-vs-actual plot from any model with `.predict()`.

Backend	Models
statsmodels	OLS, GLM, any with `predict()`
sklearn	Any regressor with `predict()`
torch	`NeuralNetFit` wrappers
keras	`NeuralNetFit` wrappers

```
import statsmodels.formula.api as smf

mod = smf.ols("y ~ x1 + x2", data=df).fit()
p = visualize(mod)
p.draw()

# With explicit data
p = visualize(mod, data=test_df, x="x1")
```

If `data=None`, extracts from `model.model.data`. Pass `x=` to choose the x-axis predictor.

COMPARE_FITS() — MODEL COMPARISON

```
compare_fits(formula, data, model1, model2, labels=["Model 1", "Model
2"], **kwargs)
```

```
m1 = smf.ols("y ~ x", data=df).fit()
m2 = smf.ols("y ~ x + z", data=df).fit()
p = compare_fits("y ~ x", data=df,
                 model1=m1, model2=m2,
                 labels=["Reduced", "Full"])
p.draw()
```

Overlays both models' predictions on the same scatter. Blue = model1, red = model2.

ADDED_PLOT() — PARTIAL REGRESSION

```
added_plot(formula, data, **kwargs)
```

Residualizes Y and X on other predictors, plots residuals against each other.

```
p = added_plot(
    "y ~ x1 + x2 + x3 | group",
    data=df)
p.draw()
```

Uses `statsmodels.OLS` internally. If no other predictors, falls back to `flexplot()`.

AUTOMATIC PLOT SELECTION

Y type	X type	Geom
numeric	numeric	scatter + smooth
numeric	categorical	jitter + mean CI
binary 0/1	numeric	GLM binomial smooth
categorical	categorical	jittered scatter
numeric	None (~1)	histogram

Color variable → `aes(color=, group=)` added to all geoms.

Neural Networks & Stats

NEURALNETFIT — WRAPPER CLASS

```
NeuralNetFit(model, response_var: str, predictor_names: List[str],
backend: str = "torch")
```

Wraps a `torch.nn.Module` or `keras.Model` for use in `visualize()` and `compare_fits()`.

```
import torch
from pyflexplot.flex_nn import NeuralNetFit

net = torch.nn.Sequential(
    torch.nn.Linear(3, 8),
    torch.nn.ReLU(),
    torch.nn.Linear(8, 1)
).eval()

fit = NeuralNetFit(
    model=net,
    response_var="y",
    predictor_names=["x1", "x2", "x3"]
)

# Use like any model
p = visualize(fit, data=df)
p = compare_fits("y ~ x1", df, m1, fit)
```

PERMUTATION_IMPORTANCE()

```
permutation_importance(fit, X, y, metric=None, n_repeats=1,
random_state=None, higher_is_better=None)
```

Column-shuffling variable importance. Returns a DataFrame ranked by importance.

```
result = permutation_importance(
    fit, X=df[["x1","x2","x3"]],
    y=df["y"],
    metric="mse"
)
print(result.sort_values(
    "importance", ascending=False))
```

metric	Scoring
`"mse"` / `"loss"`	Mean squared error
`"r2"`	R-squared
`"auc"`	Rank-based AUC
`"accuracy"`	Classification accuracy
`"precision"` / `"recall"` / `"f1"`	Binary metrics

BIOSTATS UTILITIES (FIFER)

```
model_comparison(model1, model2) estimates(model) p_format(p, digits=3)
eliminated_columns(df, threshold=0.5) color_table(df, cmap="viridis")
```

```
m1 = smf.ols("y ~ x1", data=df).fit()
m2 = smf.ols("y ~ x1 + x2", data=df).fit()
model_comparison(m1, m2)

estimates(m2)
p_format(0.0341, digits=3)  # "p = .034"
eliminated_columns(df, threshold=0.8)
color_table(df.corr(), cmap="coolwarm")
```

SEM VISUALIZATION (SEMOPY)

```
hopper_plot(model, **kwargs) disturbance_plot(model, var1, var2, data)
measurement_plot(model, latent_var, indicator, data)
```

```
from semopy import Model
mod = Model("F1 =~ x1 + x2 + x3")
mod.fit(df)
hopper_plot(mod).draw()
disturbance_plot(mod, "F1", "F2", df)
measurement_plot(mod, "F1", "x1", df)
```

Bluepill & Empirical Bayes

MIXED_MODEL() — SYNTHETIC DATA

```
mixed_model(fixed, random, sigma, clusters, n_per, vars,
interactions=None, polynomials=None, seed=None) -> pd.DataFrame
```

```
from pyflexplot import mixed_model

df = mixed_model(
    fixed=[0.0, 0.2, 0.5, 0.3, 0.2],
    random=[0.1, 0.1, 0.0, 0.2, 0.1],
    sigma=0.3,
    clusters=15,
    n_per=[11, 3],
    vars={
        "depression":  (10.0, 3.0, 0),
        "stress":      (22.0, 7.0, 0),
        "life_events":  ["no", "yes"],
        "ses":          (55.0, 15.0, 0),
        "therapist":    [f"Dr. {chr(65+i)}"
                         for i in range(15)],
    },
    interactions={
        "stress": ["depression", "ses",
                   {"from": 0.1, "to": 0.5,
                    "coef": 0.3}],
    },
    polynomials={
        "depression": {"from": 1, "coef": 0.05},
    },
    seed=42,
)
```

Parameter	Type
`fixed`	Fixed effects coefficients
`random`	Random effect SDs
`vars`	Dict: name → (mean, sd, 0) or [levels]
`clusters`	Number of clusters
`n_per`	Rows per cluster

ESTIMATE_SD() — RECOVER SD

```
estimate_sd(mean, min_val, max_val, num_sds=3) -> float
```

```
sd = estimate_sd(mean=100, min_val=70,
                 max_val=130, num_sds=3)
# sd = 10.0
```

EMPIRICAL BAYES (EBBR)

```
fit_beta_prior(successes, totals, method="mle") add_ebb_estimate(df,
success_col, total_col, prior=None)
```

```
prior = fit_beta_prior(
    successes=df["hits"],
    totals=df["at_bats"])

df = add_ebb_estimate(
    df, "hits", "at_bats",
    prior=prior)
```

Returns a BetaPrior with `.alpha` and `.beta`. `add_ebb_estimate` adds posterior mean and credible intervals to the DataFrame.

MODULE MAP

Module	Functions
`core.py`	flexplot, visualize, compare_fits, added_plot
`stats.py`	model_comparison, estimates, p_format, eliminated_columns, color_table
`sem.py`	hopper_plot, disturbance_plot, measurement_plot
`flex_nn.py`	NeuralNetFit, permutation_importance
`bluepill.py`	estimate_sd, mixed_model
`ebbr.py`	fit_beta_prior, add_ebb_estimate

TESTING & CI

Workflow	Covers
`python-app.yml`	Core (3.10–3.13, no optional deps)
`torch.yml`	torch flex_nn tests (weekly)
`keras3.yml`	keras[jax] tests (weekly)

164 tests total. Optional backend tests `importorskip` cleanly.